

QIYAS Full-Stack Development Hackathon 2026

From Idea to Working Prototype in 10 Hours

Organizer: Addis Ababa University – College of Technology and Built Environment (CTBE)

Programme: QIYAS Advanced Digital Skills Training Project

Participants: QIYAS Full-Stack Software Development Trainees

Team Structure: 4 to 5 developers per team

Format: Onsite 10h sprint

1. What This Hackathon Is About

The QIYAS Full-Stack Hackathon tests what you actually learned over the past weeks of intensive training. You have 10 hours to take a real problem, build a working prototype from scratch, and defend your code in front of working engineers and university faculty.

Every team picks one of three challenges released at kickoff. Your job is to deliver a functional Minimum Viable Product (MVP) where a real user can complete a real task from start to finish without the app falling apart.

Here is the development path you will work through during the day:

Problem Analysis → Wireframing → Schema Design → API & Backend → Angular Frontend → Full-Stack Integration → Testing & Bug Fixes → Live Demo

2. The Three Challenges

Teams receive all three challenge briefs at the start of the event and must select one within the initial milestone window. Once you pick, stick with it; pivoting halfway through the day burns hours you cannot get back.

These three are the same weight. Same two roles, same kind of day for a team of four or five, same number of screens, same size of MVP. Each one has a **hard rule that must run on the server**, not in the Angular template, not in a judge's imagination. Judges score the required journey, not how many extra pages you invented.

Requirement	SkillBridge	FixMyCampus	EkubCircle
Two roles	Candidate / Employer	Reporter / Admin	Member / Organizer
Filterable list	Jobs by skill	Tickets by building/status	Rounds by number/status
Create form	Job + candidate profile	Issue ticket	Circle + add members
“My stuff” (Role A)	My applications	My tickets	My payments & receipt status
Dashboard (Role B)	Applicants + match % + status	Assign + move status	Record pay, payout, next round
Hard server rule	Match % + block duplicate applies	Status only moves forward	Pot paid once per member; continue paying

Refresh is enough on every challenge. Live push (SignalR) is extra credit, never a scoring gate.

2.1 Challenge 1: SkillBridge

Digital Internship & Job Matching for Tech Talent

The Problem: Graduating students and junior developers in Ethiopia struggle to find internships and entry jobs that match their actual technical skillset. Opportunities are scattered across Telegram channels, LinkedIn posts, personal word-of-mouth, and job boards with no skill verification. Meanwhile, hiring managers get flooded with hundreds of unvetted CVs and waste hours hunting for candidates who know the specific stack they need.

What You Need to Build: A matching platform where candidates tag real skills, employers post jobs with required skills, and the **API** computes how well each applicant fits.

Key User Flows:

- **Candidate:** Sign in, keep a skill-tagged profile, browse and filter jobs, apply once per job, see my applications and their status.

- **Employer:** Sign in, post a job with required skills, open the applicant list with a match percentage, set Received / Shortlisted / Rejected.

Hard rule (server): Match percentage = overlapping skills ÷ the job's required skills, as a whole number. A candidate cannot apply to the same job twice. Computing the percentage only in the browser is a miss.

Baseline MVP Requirements:

1. Authentication with Candidate and Employer roles.
2. Candidate profile with a list of technical skill tags.
3. Employer job form: title, description, required skill tags.
4. Job list the candidate can filter by skill.
5. Apply. Persist the application. Reject a second application on that job (400).
6. Candidate “my applications” with current status.
7. Employer dashboard: applicants per job, **match percentage from the API**, profile skills visible.
8. Employer can set application status: Received → Shortlisted or Rejected.

Good Ideas for Extra Credit:

- Employer can filter applicants to “meets every required skill” vs partial match.
- Link a GitHub profile or a deployed project on the candidate card.
- Withdraw an application if it is still Received.
- A short “why this match” line (which skills overlapped).

2.2 Challenge 2: FixMyCampus

Campus Issue Reporting & Maintenance Tracker

The Problem: University campuses are large, and things break every single day: lab computers crash, Wi-Fi access points die, projectors stop working, pipes leak, and power outlets short out. Right now, students and staff report issues by telling a lab assistant in person, making a phone call, or posting in a chat group. Issues get lost, nobody knows who is working on what, and university administrators have zero data on recurring facility bottlenecks.

What You Need to Build: A ticket app where reporters file a located issue and an admin assigns it and walks it **forward** through a short, fixed lifecycle.

Key User Flows:

- **Reporter (student or staff):** Sign in, submit a ticket (category, building, room, description), see the campus feed, open a ticket, see **my tickets** and the current state.
- **Admin:** Sign in, filter the list, assign a ticket to a technician by name, move status, see the timestamped history.

Hard rule (server): Status only moves forward on this path, and you cannot skip:

New → Assigned → In Progress → Resolved

Assign is what takes a ticket from New to Assigned. Resolve is illegal unless the ticket is In Progress. A dropdown that jumps New → Resolved is a miss.

Baseline MVP Requirements:

1. Authentication with Reporter and Admin roles.
2. Ticket form: category, building, room, description.
3. Campus feed with a filter on building or status.
4. Ticket detail: current state and a history of status changes with timestamps.
5. Reporter “my tickets.”
6. Admin dashboard of open tickets.
7. Admin assigns a technician by name and then moves status along the path.
8. API rejects an illegal skip or a backward move (400).

Good Ideas for Extra Credit:

- High / Medium / Low urgency (does not replace the status path).
- Building list as a dropdown so rooms are not free-typed chaos.
- Reporter confirms “this is actually fixed” after Resolved.
- Admin counts: open tickets by building.

2.3 Challenge 3: EkubCircle

Ledger for a rotating Ekub circle

The Problem: Ekub already runs in notebooks and Telegram groups across the country. A circle agrees a fixed contribution. Each meeting, everyone pays the same amount into the pot, and **one** member takes the pot. People who have already received are out of the running. They still pay. The circle ends when every member has received once. That is where memory fails: who paid, who is late, who already took the pot, and whether someone stopped paying after they received.

What You Need to Build: A ledger for **one** circle. The organizer runs rounds. Members see their own payments and whether they have received. The **API** enforces the Ekub rules. This is a record book, not a bank: no Telebirr, no CBE, no live money.

Key User Flows:

- **Member:** Sign in, see the circle (contribution, members, current round), see whether I have paid this round, see whether I have already received the pot, see who received past rounds.
- **Organizer (also a member):** Sign in, start the circle (locks the member list), mark who paid this round, pay out to the rightful person, open the next round. The organizer pays and can receive, like everyone else.

Hard rule (server): A member may receive the pot **at most once**. After they receive, they still appear on the payment list (can still pay). Payout is allowed only when every member is marked paid for that round. The receiver is the next name in the **fixed order set at Start**, not a name typed into a box. Paying the same member a second time is 400. A draw/lottery is extra credit, not the MVP.

Circle and round states:

- Circle: Forming → Active → Completed (completed = every member has received once).
- Round: Open → Paid out.

Baseline MVP Requirements:

1. Authentication. The organizer is also a member.
2. Create a circle: name, contribution in Birr, meeting label (weekly or monthly, a label, not a scheduler). Add members while it is forming.

3. **Start** locks the member list, creates one round per member, and sets a fixed payout order.
4. Round screen: who is paid / unpaid, $\text{pot} = (\text{paid count}) \times \text{contribution}$.
5. Organizer records who paid this round.
6. Member home: paid this round or not, received or not, pot so far, history of winners.
7. Organizer pays out the current round, then opens the next. API blocks a second payout to the same member, a payout before everyone has paid, and a payout on a round that is already paid out.
8. Round history: the member can filter (by round number or Paid out / Open). Amounts are recorded figures only.

Good Ideas for Extra Credit:

- Server-side draw among members who have not received yet (winner must come from the API, not a hardcoded name).
- Late flag: unpaid members cannot win a draw that round.
- A fixed late fine (even a 50 Birr note on the payment row).
- Completed-circle summary: who received which round.

Keep out of the MVP: payment providers, SMS, interest, bidding for the pot, five circles, PDF statements.

3. Technology Stack & Technical Rules

To keep the competition fair and directly test the skills taught in class, every single team must build with the official curriculum stack. If a team submits a project in another tech stack, that project will receive zero points for technical implementation.

Mandated Technologies

- **Frontend:** Angular (v16+) with TypeScript. Use Angular components, services, reactive forms, and routing. Do not use React or Vue.
- **Backend:** ASP.NET Core Web API in C# (.NET 8 or 9). Build RESTful endpoints returning proper JSON and HTTP status codes.

- **Database & Data Access:** Entity Framework Core (EF Core) code-first or migrations, paired with a relational database (SQL Server, LocalDB, PostgreSQL).
- **HTTP Communication:** Connect Angular to your API via Angular HttpClient. No hardcoded mock JSON arrays in the frontend; data must come over HTTP from your ASP.NET Core endpoints.
- **Version Control:** Git. All work must be committed to your team's designated repository throughout the day.

Architectural Standards Judges Will Check

- **Controllers Stay Slim:** Business logic belongs in service classes or handler methods, not stuffed directly into controller action methods.
- **Use DTOs:** Do not expose your raw EF Core database entities straight to frontend JSON. Map incoming and outgoing data with request and response models.
- **Proper HTTP Verbs & Codes:** Use GET for reads, POST for creations, PUT/PATCH for updates, and DELETE for removals. Return 201 Created with a location header when saving new records, 400 Bad Request for invalid input, 404 Not Found when records do not exist, and 200 OK for normal responses.
- **Client-Side Validation:** Put clear validation on your Angular forms so users see immediate feedback if they leave required fields empty.
- **Authentication & Roles:** For challenges requiring multiple user types, protect routes on the frontend with Angular guards and protect endpoints on the backend with authorization attributes.

Standard Repository README

Every team repository must include a root README.md filled out using this exact structure:

[Team Name] — [Selected Challenge Name]

Team Members & Responsibilities

- [Name] — Team Lead & Pitch
- [Name] — Frontend Developer
- [Name] — Backend Developer
- [Name] — Database & Full-Stack Developer
- [Name] — QA & Testing

Tech Stack Used

- Frontend: Angular [\[Version\]](#)
- Backend: ASP.NET Core Web API [\[.NET Version\]](#)
- Database: EF Core with [\[SQL Server / PostgreSQL / SQLite\]](#)

How to Run Locally

Backend Setup

1. cd backend
2. dotnet restore
3. dotnet ef database update
4. dotnet run

Frontend Setup

1. cd frontend
2. npm install
3. ng serve

Test Accounts & Demo Credentials

- Admin / Staff User: admin@hackathon.local / Admin123!
- Standard User: user@hackathon.local / User123!

Working Features

- List the features that actually work end to end during the demo.

Known Limitations & Bugs

- Be honest about features you started but could not finish in 10 hours.

4. Schedule & Key Checkpoints

Time	Milestone / Event	Deliverables & Focus
08:30	Hackathon Kickoff	Challenge briefs released to all teams.

Time	Milestone / Event	Deliverables & Focus
09:30	Challenge Lock	Final selection of 1 of the 3 challenges.
10:30	Checkpoint 1: Architecture & Scope Lock	Show DB schema (3–4 tables), wireframes, and team role assignments.
13:30	Checkpoint 2: Integration Check	Show active Git commits, running DB with seeds, and 1 working HTTP request between Angular & API.
16:00	Checkpoint 3: Feature Freeze	Halt new feature code; focus exclusively on bug fixes, stabilization, and demo prep.
17:30	Code Submission Deadline	Repository tag & README form submission due.

5. How to Organize Your 5-Person Team

Every person on the team needs a clearly defined daily assignment. Do not have five people crowding around one laptop watching one person type.

Role	Core Job During the 10 Hours
Team Lead / Product Owner	Keeps everyone on schedule, prevents scope creep, tests features as merged, and manages pitch narrative.
Frontend Developer	Builds Angular components, templates, styling, and reactive forms with input validation.
Backend Developer	Builds ASP.NET Core controllers, service classes, DTOs, and handles HTTP status codes.

Role	Core Job During the 10 Hours
Database & Full-Stack	Sets up EF Core models, migrations, seed data, and assists in connecting backend endpoints to database tables.
QA / Integrator	Tests forms with bad inputs, verifies edge cases, checks error messages, writes README, and verifies demo steps.

7. Scoring Rubric (100 Total Points)

Judges score every project against a 100-point standard rubric. Working code that solves the core problem is worth far more than fancy designs that do not actually work.

#	Evaluation Area	Points	What Judges Look For
1	Problem Relevance & Scope	10	Understanding of real-world problem domain & core user needs.
2	Functional Completeness of MVP	25	End to end working primary user flow and enforced hard rules.
3	Technical Architecture & Stack	20	Clean .NET + Angular compliance, slim controllers, DTOs, proper schema.
4	UI/UX & Usability	10	Clean, responsive layout, intuitive navigation, form feedback.
5	Meaningful Innovation	10	Practical working bonus feature or thoughtful algorithm addition.
6	Code Quality & Git Practices	10	Readable code, clean structure, distributed multi-author Git commits.
7	Error Handling & Reliability	5	Graceful failure on bad inputs, friendly messages, zero crashes.

#	Evaluation Area	Points	What Judges Look For
8	Teamwork & Code Defense	5	Ability of non-presenting members to explain code and design choices.
9	Live Demonstration & Pitch	5	Focused live software walk-through demonstrating working code.
Total Possible Score		100	-
Total			

8. Detailed Scoring Breakdown

8.1 Problem Relevance & Scope (10 Points)

- **Excellent (9–10):** The team thoroughly understands the problem domain. Their features match what real users would need first.
- **Good (7–8):** Clear grasp of the problem with minor scope misjudgments.
- **Satisfactory (5–6):** Basic idea is there, but the application addresses secondary issues while missing obvious core needs.
- **Weak (0–4):** The team built something unrelated to the prompt or failed to grasp the user problem.

8.2 Functional Completeness (25 Points)

This is the single biggest scoring category. Judges test your application live.

- **Excellent (22–25):** The primary workflow works cleanly from beginning to end.
SkillBridge: job posted → browse → apply once → employer sees match % and sets status. FixMyCampus: ticket filed → assigned → in progress → resolved, with an illegal skip rejected. EkubCircle: all four paid → pot paid once in order → next round, second payout rejected.
- **Good (18–21):** The primary workflow works with minor bugs or rough edges that do not prevent completion.

- **Satisfactory (13–17):** Individual screens work, but the connection breaks halfway through the workflow.
- **Weak (0–12):** Mostly static screens, hardcoded mock responses, or the app crashes during the basic user journey.

8.3 Technical Architecture & Stack Compliance (20 Points)

- **Excellent (17–20):** Built strictly with Angular and ASP.NET Core Web API with EF Core. Clear separation between controllers, services, and models. Proper HTTP verbs and status codes used everywhere. Clean relational database schema.
- **Good (13–16):** Fully compliant stack with sound design, but minor shortcuts (e.g., business logic placed directly in controllers, minor input validation gaps).
- **Satisfactory (9–12):** Compliant stack, but messy architecture (everything crammed into one file, inconsistent endpoint naming, hardcoded connection strings).
- **Weak (0–8):** Non-approved stack used, missing database layer, or frontend completely unhooked from backend.

8.4 UI/UX & Usability (10 Points)

- **Excellent (9–10):** Clean layout, clear buttons, responsive form validation messages, and visible loading indicators when fetching data.
- **Good (7–8):** Usable and clean with minor visual inconsistencies.
- **Satisfactory (5–6):** Confusing navigation, missing form labels, or user gets stuck without knowing what to click next.
- **Weak (0–4):** Broken layout, overlapping text, or unusable form controls.

8.5 Innovation & Value Addition (10 Points)

- **Excellent (9–10):** Thoughtful, practical additions that make the product significantly better (e.g., sort applicants by match, reporter confirms a fix, server-side Ekub draw, clean status badges). Do not score a team higher just because their challenge has a longer catalogue.
- **Good (7–8):** One or two solid bonus features that work.
- **Satisfactory (5–6):** Attempted extra features that are half-broken or purely cosmetic.

- **Weak (0–4):** Bare-minimum effort with zero creative problem-solving.

8.6 Code Quality & Git History (10 Points)

- **Excellent (9–10):** Readable code, clean component naming, descriptive commits spread across all team members from 10:00 to 17:00, and a helpful README.
- **Good (7–8):** Solid code organization with regular commits, minor stylistic inconsistency.
- **Satisfactory (5–6):** Sloppy variable names, sparse commits, or one person committed everything in a single bulk push.
- **Weak (0–4):** Chaotic repository, unreadable copy-pasted code, or no Git history.

8.7 Error Handling & Reliability (5 Points)

- **Excellent (5):** Handles empty inputs and invalid values cleanly with friendly error messages. Zero crashes during judging.
- **Good (4):** Handles common mistakes without throwing unhandled exceptions.
- **Satisfactory (3):** Works fine on happy path, but crashes or shows raw yellow-screen stack traces when invalid data is entered.
- **Weak (0–2):** App crashes repeatedly during basic testing.

8.8 Teamwork & Code Defense (5 Points)

- **Excellent (5):** Every team member can explain their section of the code clearly and confidently.
- **Good (4):** Most members understand the code and can answer questions.
- **Satisfactory (3):** Work was clearly done by only one or two people; other members cannot explain how the code works.
- **Weak (0–2):** Only one person knows anything about the repository; others cannot answer basic questions.

8.9 Live Demonstration & Pitch (5 Points)

- **Excellent (5):** Direct, professional demo showing working software immediately. Clear answers to judge questions.
- **Good (4):** Good live demo with minor narrative stumbles.
- **Satisfactory (3):** Spent too much time talking about slides instead of showing working code.
- **Weak (0–2):** Could not get the demo running on stage or could not answer basic technical questions.

9. Final Stage Pitch Format (Top 6 Teams)

Finalist teams get an **8-minute stage slot** in the main auditorium:

- **5 Minutes:** Presentation and live working demonstration.
- **3 Minutes:** Technical questions from the Grand Final judging panel.

Suggested 5-Minute Time Breakdown

- **Minute 0:00 – 0:30:** State the problem and your solution concept in simple plain English.
- **Minute 0:30 – 1:00:** Show your architecture: mention your Angular frontend, API endpoints, and database schema.
- **Minute 1:00 – 4:00: Live Product Walkthrough.** Show the user journey working live on screen. Do not show screenshots or Figma slides.
- **Minute 4:00 – 4:30:** Highlight your best innovation feature or algorithm.
- **Minute 4:30 – 5:00:** Be honest about known limitations, what broke, and what you would build with another week.

10. Competition Rules & Academic Honesty

- 1. Eligibility:** Participants must be active trainees in the QIYAS Full-Stack programme. Teams have 4 to 5 members.
- 2. Fresh Development:** Every line of application code must be written during the 10-hour competition window. Submitting existing school projects, previous

assignments, or commercial work is strictly prohibited and results in immediate disqualification.

- 3. Allowed Assets:** Framework CLI generators (ng new, dotnet new webapi), standard npm packages, NuGet packages, public documentation, and open APIs are completely fine. Pre-built template applications are not allowed.
- 4. Git Verification:** All code must be pushed to the official team Git repository. Judges will audit commit timestamps.
- 5. Code of Conduct:** Respect fellow participants, mentors, and judges. Cheating, copying code from another team, or disruptive behavior results in immediate removal from the venue.

11. AI Tools & Mandatory Code Defense

You are allowed to use AI coding assistants for:

- debugging errors,
- writing boilerplate syntax,
- looking up documentation.

However, you must be able to defend your code:

- During Stage 1 desk visits, judges will ask **at least one non-presenting team member** to explain how a specific API endpoint works, trace a database query, or explain a TypeScript function.
- If your team submits complex code generated by an AI assistant that your own members cannot explain or modify, your team will receive major point deductions under Code Quality and Teamwork.

12. Final Submission Checklist

At the end, your team lead must submit the official form containing:

- ☐ Working Git repository URL with all code committed.
- ☐ Completed README.md following the template in Section 3.
- ☐ Working application launch steps verified on localhost.
- ☐ Clean database seed script so judges can test immediately.

13. Awards & Prizes

Championship Prizes

- **1st Place — QIYAS Champion: 30,000 ETB**
- **2nd Place — First Runner-Up: 20,000 ETB**
- **3rd Place — Second Runner-Up: 10,000 ETB**

14. Mentor Rules & Support

Technical mentors will be present throughout the build period:

- Mentors can:
 - help you understand challenge requirements,
 - advise on cutting scope if you are running out of time,
 - and teach you how to troubleshoot bugs.
- **Strict Mentor Rule:** Mentors are not allowed to type code, write database queries, or solve architectural problems for you. Mentors guide your hands; they do not build your project.
- Each mentor oversees 4 to 5 teams to make sure help is evenly distributed across the room.

15. Tie-Breaking Sequence

If two or more teams end with identical total scores, the tie is broken using this exact priority order:

1. Highest score in **Functional Completeness of MVP** (Criterion 2).
2. Highest score in **Technical Architecture & Implementation** (Criterion 3).
3. Highest score in **Problem Relevance & Understanding** (Criterion 1).
4. If still tied, the tied teams participate in an impromptu 3-minute technical Q&A session with the Grand Final jury.

